

Rapport de Conception : Systèmes Multi-Agents

Coalition Structure Generation et Planification Multi-Agent pour le Contrôle Dynamique d'Intersections

Application aux Pelotons de Véhicules Autonomes : Approches Centralisée et Distribuée

Présenté par : Mezaourou Sidali P2515000

Date : 04 Février 2026

Contexte : Projet de Recherche et Développement en Intelligence Artificielle

Domaines : Formation de Coalitions (CSG), Planification Automatique, Systèmes de Transport Intelligents (ITS)

Sous la supervision de : Professeur Samir Aknine

Module : Ouverture à la Recherche

Année Universitaire : 2025 - 2026

1. Introduction et Contexte du Projet

Les systèmes multi-agents constituent un paradigme majeur de l'intelligence artificielle, où plusieurs entités autonomes collaborent pour résoudre des problèmes complexes. Dans ce contexte, deux problématiques centrales émergent: la formation de coalitions (Coalition Structure Generation - CSG) et la planification multi-agent. Le CSG détermine comment partitionner un ensemble d'agents en coalitions pour maximiser une utilité globale, tandis que la planification coordonne les actions de ces coalitions pour accomplir leurs tâches.

1.1 Problématique de Recherche

Ce projet explore l'intersection entre ces deux domaines dans le contexte d'une application concrète: le contrôle dynamique d'intersections pour véhicules autonomes organisés en pelotons. La question centrale est: comment intégrer la formation de coalitions avec la planification temporelle pour optimiser la traversée d'une intersection intelligente? Le projet poursuit deux objectifs complémentaires. Premièrement, développer une approche centralisée où un contrôleur global calcule la configuration optimale de pelotons et le plan de traversée pour maximiser l'utilité du système. Deuxièmement, développer une approche distribuée où les véhicules négocient localement pour former des pelotons et coordonner leurs traversées de manière décentralisée. Ces deux approches représentent des compromis différents entre optimalité et scalabilité.

1.2 Contributions Attendues

Les contributions de ce travail sont à la fois théoriques et pratiques. Sur le plan théorique, nous proposons une formalisation unifiée du problème de formation de pelotons comme un problème de CSG avec contraintes spatio-temporelles. Je conçois un nouvel algorithme (C-CSG-Plan) fusionnant génération de coalitions optimale et planification temporelle. Sur le plan pratique, je fournis une implémentation dans un simulateur de trafic et une évaluation comparative rigoureuse des approches centralisée et distribuée.

2. État de l'art : CSG et planification multi-agent

2.1 Coalition Structure Generation (CSG)

Le **CSG** consiste à partitionner un ensemble d'agents en coalitions optimales pour maximiser l'utilité globale. Le problème est NP-complet, et le nombre de structures possibles croît très rapidement avec le nombre d'agents, rendant l'énumération exhaustive impossible.

Des algorithmes avancés ont été développés pour traiter ce problème :

- **Offline Coalition Selection (OCS)** ([Taguelmimt et al., IJCAI 2024][1]) : pré-filtrage des coalitions prometteuses pour réduire l'espace de solutions.

- **Compact Solution Space Representation** ([Taguelmimt et al., IJCAI 2023][2]) : représentation compacte de l'espace de solutions pour réduire la complexité mémoire.
- **Multiagent Path Search Algorithm** ([Taguelmimt et al., AAAI 2025][3]) : recherche par chemins multiples avec élagage intelligent pour traiter efficacement des instances de grande taille (jusqu'à 25 agents).

Ces algorithmes constituent la base de l'approche centralisée C-CSG-Plan.

2.2 Planification multi-agent

La planification multi-agent organise les séquences d'actions pour atteindre les objectifs. Elle peut être :

- **Centralisée**, avec un planificateur global qui calcule le plan complet, offrant une solution optimale mais peu scalable.
- **Distribuée**, où les agents planifient localement et se coordonnent, plus flexible mais potentiellement sous-optimale.

2.3 Lacunes et positionnement

Bien que CSG et planification multi-agent soient largement étudiés séparément, leur intégration conjointe reste peu explorée. Le projet vise à coupler formation de pelotons et planification temporelle pour améliorer la coordination et l'efficacité dans le trafic de véhicules autonomes.

3. Scénario d'Application : Pelotons de Véhicules Autonomes

3.1 Description du scénario

Le scénario concerne le contrôle dynamique d'une intersection intelligente par plusieurs pelotons de véhicules autonomes. Les pelotons permettent à plusieurs véhicules de circuler en formation serrée, améliorant l'efficacité, la fluidité et la capacité du trafic.

3.2 Opérations sur les pelotons

Le système gère trois opérations principales :

- **Fusion** : regroupe deux pelotons compatibles pour augmenter l'efficacité.
- **Éclatement** : divise un peloton lorsque les véhicules prennent des directions différentes.
- **Modification d'itinéraire** : réoriente certains pelotons pour optimiser le trafic global.

Ces opérations doivent être coordonnées avec l'allocation des créneaux de traversée à l'intersection.

3.3 Entités principales

- **Véhicules autonomes** : agents individuels avec position, vitesse, destination et niveau de coopération.
- **Pelotons** : coalitions de véhicules avec leader et trajectoire commune.
- **Intersection intelligente** : coordonnateur allouant les créneaux et arbitrant le trafic.
- **Contrôleur central** : planificateur global (approche centralisée) pour optimiser la formation de pelotons et l'ordonnancement.

4. Approche Centralisée : Algorithme C-CSG-Plan

4.1 Principe de l'Approche Centralisée

Cette étude commence par une approche centralisée afin d'établir une baseline optimale. Dans cette configuration, le contrôleur d'intersection centralise toutes les informations des véhicules et calcule globalement :

- La configuration optimale des pelotons (coalitions),
- Le plan de traversée de l'intersection.

Cette approche garantit l'optimalité, sous réserve de temps de calcul suffisant, et sert de borne supérieure pour évaluer l'approche distribuée.

Note : Il n'y a pas de négociation entre véhicules dans cette approche. Les véhicules transmettent leurs caractéristiques (position, vitesse, destination, priorité) au contrôleur, qui prend toutes les décisions computationnellement. L'argumentation et la négociation sont réservées à l'approche distribuée.

4.2 Architecture de l'Algorithme C-CSG-Plan

L'algorithme C-CSG-Plan (Centralized Coalition Structure Generation and Planning) combine deux composantes complémentaires :

1. **Génération de coalitions (CSG)** : adapte l'algorithme de recherche par chemins multiples de Taguelmimt et al., AAAI 2025 pour générer la configuration optimale des pelotons.
2. **Planification temporelle** : effectue l'ordonnancement des traversées en tenant compte des conflits directionnels.

Ces deux composantes sont couplées itérativement pour converger vers une solution globalement optimale.

4.3 Pseudo-code de l'Algorithme C-CSG-Plan

Entrée :

- $V = \{v_1, v_2, \dots, v_n\}$: ensemble des véhicules
- I : intersection avec ses caractéristiques
- t_{current} : instant courant

Sortie :

- CS^* : configuration optimale des pelotons
- τ^* : plan de traversée (allocation de créneaux)

Phase 1 : Collecte et Pré-traitement

```
Pour chaque véhicule  $v_i \in V$  :  
    Collecter : position  $p_i$ , vitesse  $vel_i$ , destination  $d_i$ , coopération  $c_i$ ,  
    priorité  $w_i$   
    Calculer  $ETA_{\text{naturel}}(v_i, I)$  // temps d'arrivée sans modification
```

Phase 2 : Génération de coalitions candidates

```
 $C_{\text{candidates}} \leftarrow \text{GenerateCandidatePlatoons}(V)$   
Pour chaque coalition  $P \in C_{\text{candidates}}$  :  
    Si  $\neg \text{SatisfiesConstraints}(P)$  : // contraintes spatiales / trajectoire  
        Supprimer  $P$  de  $C_{\text{candidates}}$   
    Sinon :  
        Calculer utilité  $u(P)$ 
```

Phase 3 : Résolution CSG optimale

```
 $CS^* \leftarrow \text{MultiagentPathSearch}(C_{\text{candidates}}, V)$   
// Taguelmimt et al. (AAAI 2025) avec représentation compacte et élagage
```

Phase 4 : Planification temporelle et ordonnancement

```
Trier les pelotons de  $CS^*$  par priorité décroissante : transports en commun >  
véhicules prioritaires > autres  
 $\tau^* \leftarrow \emptyset$   
 $timeline \leftarrow \text{InitializeTimeline}(I)$  // créneaux libres  
  
Pour chaque peloton  $P \in CS^*$  :  
     $t_{\text{arrival}} \leftarrow \text{EstimateArrivalTime}(P, t_{\text{current}})$   
     $duration \leftarrow \text{ComputeCrossingDuration}(P, I)$   
  
     $slot_{\text{optimal}} \leftarrow \text{FindBestSlot}(timeline, P, t_{\text{arrival}}, duration)$   
  
    Si  $slot_{\text{optimal}}$  existe :  
         $\tau^*(P) \leftarrow slot_{\text{optimal}}.start\_time$   
         $\text{ReserveSlot}(timeline, slot_{\text{optimal}})$   
         $\Delta vel \leftarrow \text{ComputeSpeedAdjustment}(P, t_{\text{arrival}}, \tau^*(P))$   
        Si  $|\Delta vel| \leq vel\_adjustment\_max$  :  
             $\text{ApplySpeedAdjustment}(P, \Delta vel)$ 
```

```

Sinon :
    slot_alternatif ← FindNextAvailableSlot(timeline, duration)
     $\tau^*(P) \leftarrow \text{slot\_alternatif.start\_time}$ 
    ReserveSlot(timeline, slot_alternatif)
Sinon :
     $CS^*, \tau^* \leftarrow \text{SplitPlatoonAndReschedule}(CS^*, \tau^*, P)$ 

```

Phase 5 : Vérification et optimisation finale

```

Pour chaque paire de pelotons (P1, P2) avec directions conflictuelles :
    Si IntervalOverlap( $[\tau^*(P1), \tau^*(P1) + \Delta t(P1)]$ ,  $[\tau^*(P2), \tau^*(P2) + \Delta t(P2)]$ ) :
        Résoudre conflit par réordonnancement local

Retourner ( $CS^*, \tau^*$ )

```

4.4 Fonctions Auxiliaires Clés

- **GenerateCandidatePlatoons(V)** : génère les coalitions candidates avec filtrage heuristique pour respecter les contraintes de proximité spatiale, compatibilité de trajectoire, et taille maximale.
- **MultiagentPathSearch(C_candidates, V)** : implémente l'algorithme de recherche par chemins multiples de **Taguelmimt et al., AAAI 2025**, utilisant représentation compacte et élagage agressif. Garantit l'optimalité en pratique.
- **FindBestSlot(timeline, P, t_arrival, duration)** : recherche le créneau optimal pour minimiser l'ajustement de vitesse.
- **ComputeSpeedAdjustment(P, t_arrival, t_scheduled)** : calcule Δv_{el} pour que le peloton arrive exactement à l'heure planifiée.

4.5 Exemple d'Exécution

Scénario simplifié : 6 véhicules approchant une intersection en croix

- v1, v2, v3 : du Nord vers le Sud
- v4, v5 : de l'Est vers l'Ouest
- v6 : du Sud vers le Nord

Phase 2 - Coalitions candidates générées

$C_candidates = \{\{v1, v2, v3\}, \{v4, v5\}, \{v6\}, \dots\}$ après filtrage des contraintes

Phase 3 - Résolution CSG

$CS^* = \{\{v1, v2, v3\}, \{v4, v5\}, \{v6\}\}$

Utilité totale = 14.2 (2 pelotons + 1 véhicule isolé)

Phase 4 - Ordonnancement

1. $P1 = \{v1, v2, v3\} : \tau^*(P1) = 10s, \text{durée } 3s \rightarrow [10s, 13s]$
2. $P2 = \{v4, v5\} : \tau^*(P2) = 14s, \text{durée } 2s \rightarrow [14s, 16s]$
3. $P3 = \{v6\} : \tau^*(P3) = 17s, \text{durée } 1s \rightarrow [17s, 18s]$

Résultat : plan de traversée sans conflit, temps moyen = 14s (vs 18s sans pelotons)

5. Approche Distribuée: Protocoles de Négociation

5.1 Principe de l'Approche Distribuée

Dans cette approche, l'intelligence est répartie entre les véhicules autonomes. Chaque véhicule agit comme un agent rationnel, négociant localement pour former des pelotons et coordonner les traversées, sans contrôleur omniscient. La communication V2V et V2I permet une coordination émergente.

5.2 Architecture Multi-Agent Distribuée

- **Agents Véhicules :** Perception locale, communication, négociation pour rejoindre ou créer des pelotons.
- **Agents Pelotons :** Coordonnent les membres, négocient les créneaux et gèrent fusion/éclatement.
- **Agent Intersection :** Facilite l'allocation des créneaux, arbitre les conflits, diffuse l'état local. Il ne calcule pas les pelotons optimaux.

5.3 Protocole de Négociation

1. **ANNOUNCE :** Un véhicule diffuse ses caractéristiques et préférences.
2. **PROPOSE :** Les pelotons/voitures compatibles répondent avec utilité estimée et conditions d'adhésion.
3. **Évaluation :** Le véhicule initiateur choisit la meilleure proposition selon sa fonction d'utilité personnelle.
4. **ACCEPT/REJECT :** Contrat formalisé avec le peloton choisi.
5. **Formation :** Le véhicule ajuste vitesse et position pour rejoindre le peloton.

5.4 Système d'Argumentation

Les agents utilisent des arguments pondérés pour convaincre :

- Économiques : économie de carburant
- Temporels : gain de temps
- Capacité : libération de créneaux
- Priorité : transports collectifs
- Urgence : véhicules prioritaires

L'intersection évalue ces arguments selon une grille de priorités pour garantir équité et efficacité.

5.5 Algorithme Distribué : Agent Véhicule

Entrée :

v : véhicule autonome

P : peloton actuel (null si isolé)

r : score de réputation

Environnement local : capteurs, V2V/V2I

Sortie :

Mise à jour du peloton et allocation de créneau de traversée

Phase 1 : Boucle principale (chaque Δt)

- 1 Percevoir environnement local
- 2 Mettre à jour (position p, vitesse vel)
- 3 Si $P == \text{null}$: // véhicule isolé
 - Si conditions favorables : SearchForPlatoon()
- 4 Sinon : // dans un peloton
 - MaintenirPlatoonCohesion()
 - Si approche intersection et je_suis_leader() : NegotiateCrossingWithIntersection()
 - Si trajectoire diverge : LeavePlatoon()
- 5 Exécuter action décidée

Phase 2 : Recherche de peloton (SearchForPlatoon)

1. neighbors $\leftarrow$ DetectNearbyVehiclesAndPlatoons()
2. candidates $\leftarrow$ FilterCompatibleCandidates(neighbors)
3. Pour chaque $C \in \text{candidates}$: Envoyer ANNOUNCE(caractéristiques, préférences)
4. proposals $\leftarrow$ Attendre PROPOSE(timeout)
5. Si proposals $\neq \emptyset$:
 - best $\leftarrow$ argmax_utility_personnelle(proposals)
 - Si utility_personnelle(best) > seuil :
 - Envoyer ACCEPT(best)
 - $P \leftarrow \text{JoinPlatoon}(\text{best.peloton})$
 - AjusterVitesseEtPosition()
 - Sinon : Envoyer REJECT à tous

Phase 3 : Négociation avec l'intersection (NegotiateCrossingWithIntersection)

1. $ETA \leftarrow EstimateArrivalTime(P)$
2. $arguments \leftarrow GenerateArguments(P)$
3. Envoyer REQUEST_SLOT(P, ETA, durée, arguments)
4. $proposition \leftarrow Attendre REponse(timeout)$
5. Si acceptée :
 - $creneau \leftarrow proposition.time_slot$
 - $\Delta vel \leftarrow CalculerAjustementVitesse(ETA, creneau)$
 - Si $|\Delta vel| \leq vel_max$:
 - Diffuser ADJUST_SPEED au peloton
 - Confirmer à l'intersection
 - Sinon : Renégocier ou éclater
6. Sinon : Gérer rejet (attendre ou éclater)

Phase 4 : Convergence et apprentissage

- Mettre à jour le score de réputation r
- Appliquer normes sociales et règles de priorité
- Ajuster stratégie de négociation via apprentissage par renforcement

5.6 Mécanismes de Convergence

- **Réputation** : Score $r \in [0,1]$ pour fiabilité et coopération.
- **Normes sociales** : Règles partagées (priorité bus, signalisation, distances).
- **Apprentissage par renforcement** : Ajustement des stratégies selon succès passés.
- **Arbitrage de l'intersection** : Priorité ambulance > bus > voiture, utilité totale, temps d'attente.

6. Implémentation et Architecture

6.1 Architecture Logicielle Globale

L'implémentation repose sur une architecture modulaire permettant de tester et comparer les deux approches (centralisée et distribuée) de manière indépendante tout en facilitant l'intégration avec le simulateur de trafic SUMO.

Les principaux modules sont :

- **Module CSG** : Basé sur les algorithmes de Taguelmimt et al. [1–3], ces modules implémentent la génération optimale de coalitions et étendent leurs travaux pour inclure la planification intégrée dans C-CSG-Plan.

- **Module de planification** : Gère l'ordonnancement temporel des pelotons, les ajustements de vitesse, et la résolution de conflits sur les créneaux d'intersection.
- **Module de communication** : Simule les protocoles V2V et V2I pour l'approche distribuée, incluant annonces, propositions et négociations entre véhicules et pelotons.
- **Module de visualisation** : Interface temps réel permettant d'observer l'évolution des pelotons, la formation des coalitions et la traversée des intersections.

Cette architecture modulaire garantit la réutilisabilité des composants, l'extensibilité vers des réseaux multi-intersections, et la possibilité de comparer différentes stratégies d'optimisation dans un environnement contrôlé.

6.2 Technologies Utilisées

- **Python** : Langage principal pour sa richesse en bibliothèques scientifiques et sa compatibilité avec SUMO.
- **SUMO** : Simulateur open-source de trafic microscopique pour véhicules autonomes.
- **TraCI** : Interface de contrôle en temps réel pour connecter SUMO aux algorithmes CSG et planification.
- **NetworkX** : Manipulation de graphes pour représenter les coalitions et leur structure.
- **NumPy / SciPy** : Calculs numériques, optimisation et analyse statistique.
- **Matplotlib / Seaborn** : Visualisation graphique et génération de courbes de performance.
- **ZeroMQ** : Simulation de la communication distribuée entre agents véhicules et intersection.

6.3 Protocole d'Évaluation

L'évaluation vise à comparer les performances des approches centralisée (C-CSG-Plan) et distribuée face à une baseline classique (feux tricolores).

Métriques principales :

1. **Trafic** : Temps de traversée moyen, débit, temps d'attente moyen, taux d'utilisation de l'intersection.
2. **Coalitions** : Taille moyenne des pelotons, durée de vie moyenne, stabilité et utilité par coalition.
3. **Énergie et environnement** : Consommation totale, gains relatifs vs baseline, émissions de CO₂ évitées.
4. **Équité** : Variance des temps d'attente, coefficient de Gini, respect des priorités (transports collectifs et véhicules d'urgence).

Scénarios testés : trafic léger, modéré, dense, mix de véhicules, perturbations aléatoires (arrivées imprévues, véhicules prioritaires). Chaque scénario est répété 30 fois avec des graines aléatoires différentes pour assurer une robustesse statistique.

Analyse statistique : Les résultats seront validés via tests de significativité (ANOVA ou Wilcoxon) afin de confirmer que les différences observées entre les approches ne sont pas dues

au hasard. Cette étape garantit une interprétation rigoureuse des gains en efficacité, en stabilité et en consommation énergétique.

7. Conclusion et Perspectives

Ce projet a exploré l'intégration de la formation de coalitions (CSG) et de la planification multi-agent pour les intersections intelligentes. Deux approches ont été étudiées : la centralisée (C-CSG-Plan), garantissant une solution proche de l'optimalité, et la distribuée, basée sur la négociation locale, plus scalable et résiliente.

Sur le plan théorique, cette étude formalise le problème PCSG-Plan avec des contraintes spatio-temporelles et démontré la convergence et les garanties d'optimalité conditionnelle pour chaque approche. Sur le plan pratique, l'implémentation modulaire dans SUMO permet de tester et d'évaluer en temps réel la formation et la coordination des pelotons, avec validation statistique des performances (ANOVA, Wilcoxon).

Parmi les perspectives : extension à des réseaux multi-intersections, intégration d'apprentissage par renforcement pour des stratégies adaptatives, prise en compte de l'incertitude et optimisation multi-objectifs combinant efficacité, équité et consommation. Enfin, l'étude de scénarios mixtes (autonomes et conventionnels) prépare le terrain pour un déploiement réaliste à moyen terme.

En résumé, ce travail fournit une base solide alliant innovation algorithmique, architecture modulaire et évaluation rigoureuse pour la coordination intelligente de véhicules autonomes en pelotons.



Lyon 1